Guide Ivy
Specification-based testing of QUIC

Crochet Christophe
Computer Science dep.
Université catholique de Louvain
Louvain-la-Neuve, Belgium
christophe.crochet@student.uclouvain.be

Sambon Jean-François
Computer Science dep.
Université catholique de Louvain
Louvain-la-Neuve, Belgium
jean-francois.sambon@student.uclouvain.be

***Abstract*—This document is a model and instructions for the specification-based language Ivy [1] created by Kenneth McMillan, *Odel Padon*, *Aurojit Panda*, *Mooly Sagiv* and *Sharon Shoham* applied to QUIC topic, that we used in our master thesis "*Toward-verification-of-QUIC-extensions*" subject proposed by *Bonaventure Olivier* and *Legay Axel* (and help of the networking team of UCL of course)**

***Index Terms*—QUIC, Ivy, Specification, Test, Network, Protocol, Verification**

I. Introduction

In this document, we explain the main Ivy concepts which are used for the specification-based testing of QUIC implementations.

It is based on the official documentation of Ivy where we include some specific examples to point out the concepts presented in the 1.7 version of Ivy. We only present a subset of Ivy functionalities and encourage the reader to read the official documentation for more in depth explanation.

In our thesis, we are checking if Ivy is in practice usable for protocols testing first and more specifically if it can be used to test protocol extensions with a focus on QUIC. Therefore, The goal of this document is to explain more in depth how Ivy is working from behind and how can we use this tool.

II. Ivy

"Ivy is a research tool intended to allow interactive development of protocols and their proofs of correctness and to provide a platform for developing and experimenting with automated proof techniques. In particular, Ivy provides interactive visualization of automated proofs, and supports a use model in which the human protocol designer and the automated tool interact to expose errors and prove correctness." [12] It can also extract executable programs by translation with python to C++ for efficiency. [13]

First of all, Ivy language has three kinds of object:

1) **Data values** which are the variables that we store in the state of our program.
2) **Function values** which are mathematical functions : from a given input it gives always the same output and does not modify the environment.
3) **Procedure** will modify the environment which allow different results for different execution. It can return at most one value.

We can also use high-order programming with the Ivy language by putting into variables data values and function values. There are no references in Ivy, two variables can't contain the same object.

The language is synchronous, which means that all actions occur in reaction to changes in the environment and that they are also all isolated, so they appear to occur instantly.

These features are present since most of verification problems such as invariant checking, model checking rely on a subset of first-order logic formulas: decidable fragments. [5] More details will be given in the section V.

We will explain the Ivy concepts, and then explore the specification of the different QUIC layers, to finally introduce some concepts of the Ivy solver:

1) Application over QUIC
2) TLS handshake security
3) QUIC Frame protocol
4) QUIC Packet protocol
5) QUIC Packet Protection for reliability
6) UDP

III. Ivy concepts used for QUIC

A. Type object, Action and State

```
object frame = {                                                    1
    type this # The base type for frames                            2
    object stream = {  # Stream frames                              3
        # Stream frames are a variant of frame                      4
        variant this of frame = struct {                            5
            off : bool, # is there an offset field                  6
            len : bool, # is there a length field                   7
            fin : bool, # is this the final offset                  8
            id  : stream_id,     # the stream ID                    9
            offset : stream_pos,# the stream offset                 10
            length : stream_pos,# length of the data                11
            data : stream_data  # the stream data                   12
        }                                                           13
    }                                                               14
}                                                                   15
```

Listing 1. State example

```
object frame = {                                                    1
    ...                                                             2
    action handle(f:this,    # param_name:type                    3
                  scid:cid,                                         4
                  dcid:cid,                                         5
                  e:quic_packet_type) = {                           6
       # this generic action should never be called                 7
       require false;                                               8
    }                                                               9
}                                                                   10
action enqueue_frame(scid:cid,                                      11
                     f:frame,                                       12
                     e:quic_packet_type,                            13
                     probing:bool) = {                              14
       # Code                                                       15
}                                                                   16
```

Listing 2. Action example

```
object frame = {                                                    1
    ...                                                             2
    object stream = {                                               3
        ...                                                         4
        action handle(f:this,                                       5
                      scid:cid,                                     6
                      dcid:cid,                                     7
                      e:quic_packet_type) = {                       8
           require connected(dcid)                                  9
                   & connected_to(dcid)=scid;                       10
           require e = quic_packet_type.one_rtt                     11
                     & established_1rtt_keys(scid);                 12
           call enqueue_frame(scid,f,e,false);                      13
        }                                                           14
    }                                                               15
}                                                                   16
```

Listing 3. Action example

In this example, we can already see the most used concepts of Ivy for testing QUIC protocols. First, Ivy use the concept of `type objects` which have a **state** (1) that can be manipulated through **action** (2,3).

This example also show you that type objects are defined from a very abstract and generic point of view, here we start by defining a `frame` type object, and then we define `variant` of the type object that set the state of more "embedded" structure, here with `stream frame`, and any other type of frame that QUIC defines in the specification. The same idea is use through all layers of QUIC.

Actions are also defined from an abstract and generic point of view and then adapted to specific case, here for the action `handle` which is associated to frame type object and then define this action for each type of frame but can also be global such as the action `enqueue_frame`. The code of the action `handle` mostly defines a set of requirements/conditions that should be true at each step to continue the execution. This set is based on the requirement specifications of the QUIC draft from the IETF.

B. Modules

Modules are like template class in object oriented programming that can be instantiated. For QUIC, it is used for defining a general class of objects but can be used to capture more general theory and proof. Here is a example for the QUIC transport parameters:

```
type trans_params_struct                                            1
object transport_parameter = {                                      2
    type this                                                       3
    action set(p:this,s:trans_params_struct)                        4
        returns (s:trans_params_struct) = {}                        5
}                                                                   6
module trans_params_ops(ptype) = {                                  7
    destructor is_set(S:trans_params_struct) : bool                 8
    destructor value(S:trans_params_struct) : ptype                 9
    action set(p:ptype,s:trans_params_struct)                       10
        returns (s:trans_params_struct) = {                         11
        is_set(s) := true;                                          12
        value(s) := p;                                              13
    }                                                               14
}                                                                   15
```

Listing 4. Transport parameter module

```
object initial_max_stream_data_bidi_local = {                       1
    variant this of transport_parameter = struct {                  2
        stream_pos_32 : stream_pos  # tag = 0                       3
    }                                                               4
    instantiate trans_params_ops(this)                              5
}                                                                   6
object initial_max_data = {                                         7
    variant this of transport_parameter = struct {                  8
        stream_pos_32 : stream_pos # tag = 1                        9
    }                                                               10
    instantiate trans_params_ops(this)                              11
}                                                                   12
```

Listing 5. Instantiating the module

In this example, we can see that the object can instantiate `trans_params_ops` automatically such that we can now check if a parameter is set or not and make call directly on transport parameter object such as:

```
initial_max_stream_data.is_set(fml:s)                               1
initial_max_stream_data.value(fml:s)                                2
initial_max_data.is_set(fml:s)                                      3
initial_max_data.value(fml:s)                                       4
trans_params(scid) := tps.transport_parameters                      5
                        .value(idx)                                 6
                        .set(trans_params(scid));                   7
```

Listing 6. Example of usage

C. Monitor

Before deeping into the specification statements, one important concept in Ivy is the monitor. The QUIC specification is based on that concept. Monitors are useful to separate the specification of an object from the object itself. Monitors are objects in Ivy that contains the specification but that doesn't execute them.
It allows to control the flow of the specifications' executions. Let's see this example :

```
around app_server_open_event {                                      1
    require conn_requested(dst,src,dcid);                           2
    require ~connected(dcid) & ~connected(scid);                    3
    ...                                                             4
    call map_cids(scid,dcid);                                       5
    call map_cids(dcid,scid);                                       6
    ack_credit(scid) := ack_credit(scid) + 1;                       7
}                                                                   8
```

Listing 7. Monitor

Let's note that around is a shortcut keyword. The content above is the same as :

```
before app_server_open_event {
    require conn_requested(dst,src,dcid);
    require ~connected(dcid) & ~connected(scid);
}
after app_server_open_event {
    call map_cids(scid,dcid);
    call map_cids(dcid,scid);
    ack_credit(scid) := ack_credit(scid) + 1;
}
```

Listing 8. Equivalence 1

We can see the use of two important keywords : `before` and `after`. What is inside the before statement will be executed each time there is an event `app_server_open_event`. We will thus in this example require that there is a connection request and that the the client's `cid` for this connection as well as the `cid` selected by the server for this connection are not already part of an active connection. The after statement is similar : it executes content each time an `app_server_open_event` is generated.
This is a shortcut of this code that produce the same result :

```
action check_connection {
    require conn_requested(dst,src,dcid);
    require ~connected(dcid) & ~connected(scid);
}
action check_mapping {
    call map_cids(scid,dcid);
    call map_cids(dcid,scid);
    ack_credit(scid) := ack_credit(scid) + 1;
}

before app_server_open_event execute check_connection;
before app_server_open_event execute a2;
```

Listing 9. Equivalence 2

```
instance pc : pcap(quic_packet,quic_deser)

action show_packet(src:ip.endpoint,
                   dst:ip.endpoint,
                   pkt:quic_packet)
import show_packet

action packet_event(src:ip.endpoint,
                    dst:ip.endpoint,
                    pkt:quic_packet) = {}

action tls_send_event(src:ip.endpoint,
                      dst:ip.endpoint,
                      scid:cid,dcid:cid,
                      data:stream_data) = {}

implement pc.handle(src:ip.endpoint,
                    dst:ip.endpoint,
                    pkt:quic_packet) {
    # print the packet
    call show_packet(src,dst,pkt);
    # infer any TLS events
    call infer_tls_events(src,dst,pkt);
    # monitor the protocol
    call packet_event(src,dst,pkt);
}
```

Listing 10. Usage of monitor

D. Control and expression

Here we define control flow statement such as sequential execution, calling actions, conditions and loops. Ivy also define expression which are represented by first order logic operator: and (`&`), or (`|`), implies (`->`), iff (`<->`) and equality (`=`) and negation (`~`). Let's see a QUIC example for that:

```
object frame = {
    ...
    object crypto = {
        ...
        action handle(f:frame.crypto,
                      scid:cid,
                      dcid:cid,
                      e:quic_packet_type)

        around handle {
            #Implication
            require num_queued_frames(scid) > 0
                    -> e = queued_level(scid);
            #negation
            require ~conn_closed(scid);
            require ~(e = quic_packet_type.zero_rtt);
            #inequality
            require (f.offset + f.length)
                    <= crypto_data_end(scid,e);
            # /!\ equality /!\
            require f.data = crypto_data(scid,e)
                .segment(f.offset,f.offset+f.length);
            ...
            # Value assigment
            var length := f.offset + f.length;
            # if statement
            if crypto_length(scid,e) < length {
                crypto_length(scid,e) := length
            };
            var idx := f.offset;
            # Loop: should not be used for initialisation
            # (placeholder concept instead) TODO
            while idx < f.offset + f.length {
                crypto_data_present(scid,e,idx) := true;
                idx := idx.next
            };
            call enqueue_frame(scid,f,e,false);
            if e = quic_packet_type.handshake {
                established_1rtt_keys(scid) := true;
            }
        }
    }
}
```

Listing 11. Crypto Frame showing most operators

The sequential execution is defined by `;` character, every time we expect another requirement or action after a statement we put this character. We can call an action with the keyword `call` which execute its code.

E. Non-deterministic choice

```
action handle_tls_extensions
    (src:ip.endpoint,
     dst:ip.endpoint,
     scid:cid,
     exts:vector[tls.extension],
     is_client_hello:bool) = {
    # We process the extensions in a message in order.
    var idx := exts.begin;
    while idx < exts.end {
        var ext := exts.value(idx);
        # For every `quic_transport_parameters` extension
        if some (tps:quic_transport_parameters) ext *> tps {
            call handle_client_transport_parameters(src,
                    dst,scid,tps,is_client_hello);
            trans_params_set(scid) := true;
        };
        idx := idx.next
    };
}
```

Listing 12. Non-determinism at the QUIC transport parameters extensions

Non-deterministic choice can be represented with `*` and used for two situations:

1) On the right-hand side of an assignment, which means that the variable can take any value possible of the type.

2) In condition to create non-deterministic branch, in the presented example this represent the fact that the extension supported by client and server are non-deterministic and said more or less "if there is at least one transport parameter available in one of TLS extension proposed, then we call the action `handle_client_transport_parameters`" meaning that the proposed extension is non-deterministic in some way since it change from implementation to implementation.

F. `require` and `ensure`

These are primitives actions of Ivy but only `require` is really used for QUIC specification. The actions `ensure` and `require` are similar in the sense that if the condition return false, then the action fails and so the execution. Ivy treats the `require` statement as an assumption and the `ensure` statement as a guarantee.

We have already seen example of `require` which is related to **pre-condition**, so here is one for `ensure` which is used to represent **post-condition**, this example has been taken from the Ivy documentation:

```
action decr(x:t) returns (y:t) = {
    require x > 0;
    y := x - 1;
    ensure y < x;
}
```

IV. QUIC REPRESENTATION

A. QUIC connection

The main file for representing the QUIC connection protocol can be found at `examples/quic/quic_connection.ivy` of the branch `quic18_client` branch of McMillan since this is the last version that work for us. By the way, note that all the comments and variable's names are not updated. This file describes and defines all states and actions that are used during the QUIC connection and is defined in function of the following sub-protocol part.

1) `examples/quic/quic_types.ivy` :

In this file we find the all the definition of elementary common type that are used in the QUIC specification such as Connection ID type `cid`, the protocol version `version`, the packet number `pkt_num` and many other ones.

2) `examples/quic/quic_frame.ivy` :

Now we define a type object to represent a general QUIC frame. Then there are definitions of all other QUIC variant frames with their updated state and an associated action on how to handle them.

3) `examples/quic/quic_packet.ivy`:

This file is very simple and contain a type object representing a packet, defined by the packet's field of the QUIC specification.

4) `examples/quic/tls_protocol.ivy` :

Describes the TLS v1.2 that we updated to 1.3 based on McMillan work. There is the handshake protocol and also the transfer of raw data. This transfer is not fully general, some features cannot be represented for now such as allowing TLS messages that span multiple TLS records. There are also a serializer (used to convert Ivy object into stream of byte inside the memory with c++) and deserializer (which is the reverse process).

5) `examples/quic/quic_transport_parameters.ivy`:

The different QUIC transport parameters are represented as different modules differentiated by their respective field and value.

B. QUIC packet protection

In `examples/quic/quic_protection.ivy` we can find representation of possible packet field that are always readable even if the packet is protected. There is also the definition of the encryption and decryption processes and all actions needed. It contains also a serializer and deserializer to interact with memory directly.

C. QUIC Ivy serializer/deserializer

Here again, there are two important files that are used to put and get states/object of the QUIC protocol (except for the preceding concept where there is also this) directly into the memory.

D. Others

There is also a representation for the UDP implementation and TLS messages and many other sub-concepts that can be linked to QUIC. We omit them in the interest of space, as the general structure of all these specification is quite similar with only a few new Ivy concepts introduced.

What we can conclude here is that Ivy is quite interesting as it is quite logical and incremental to represent any protocol specification. One create building blocks that fit into each other, quite similar in how we could do for OOP programming which abstracts the details very well. However there is usually a need for serializer/deserializer which is more procedural since we interact with memory so this counter balance the last point.

There is also the fact that code rely on so many sub-protocol specification that inspecting and knowing from where a possible mistake could come from is really difficult. May be we will rewrite everything more clearly as we are currently doing with the QUIC part.

V. Decidability algorithm and concept

Verifying and proving theorems and logical rules is very complex and time consuming tasks. This is why it is imperative to have efficient algorithm and for that, McMillan and his teams decided to use the Microsoft's Z3 theorem prover since they also developed it. [6]

A. Logic concepts

First of all, we will define some logic concepts that are needed in order to understand how Ivy works.

1) Propositional logic:

In brief, the syntax of propositional logic defines the allowable sentences. The atomic sentences consist of a single proposition symbol. Each such symbol stands for a proposition that can be true or false. We use symbols that start with an uppercase letter and may contain other letters or subscripts. Here are the basic operators:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

We can also use different rules (e.g de Morgan) and axioms to solve problems (e.g Modus ponens). For those who are interresest in the subject, you can checkout this link. [14]

2) First-order logic:

Also called predicate logic, where the main differences with propositional logic are:

1) Propositions are replaced by predicates;
2) Existential and universal quantifiers are introduced
3) It is possible to express relationships between the elements of a set in a concise way, even when the set is of infinite size

With this logic we can have expressions such as "All men are mortal, Socrate is a man so Socrate is mortal".

3) Weakest preconditions:

A solver must be able to transform the predicate from the first order logic to be able to infer the logical formula. There are two main ways to build valid deduction : weakest-precondition and strongest-postconditions. Ivy construct his proofs by using weakest liberal precondition. The most general (i.e., weakest precondition) P that satisfies [9]

$$\{P\}\ S\ \{Q\} \equiv \text{ Hoare Triple}$$

With S a statement, P the precondition and Q the postcondition of the statement. To check $\{P\}S\{Q\}$, we need to prove that $P \rightarrow wp(S, Q)$.

Here is a small resume of some of the rules:

$$wp(x := E, Q) = Q[E/x]$$

$$wp(\text{assert } P, Q) = P \wedge Q$$

$$wp(\text{assume } P, Q) = P \rightarrow Q$$

$$wp(s1; s2, Q) = wp(s1, wp(s2, Q))$$

$$wp(s1\ s2, Q) = wp(s1, Q) \wedge wp(s2, Q)$$

4) Weakest liberal preconditions:

Weakest liberal precondition is denoted as wlp. If S is a program statement and Q is a some condition on the program state, $wlp(S, Q)$ is the weakest condition P such that, if P holds before the execution of S and if S terminates, then R holds after the execution of S. [8] [10]

Again, for each triple $\{P\}S\{Q\}$, we have to prove that $P \rightarrow wps(S, Q)$ with the modified rules.

The difference with a weakest precondition wp is that in the liberal one we don't guarantee that S terminates.

5) WLP calculus and rules:

$$wlp(skip, Q) = Q \textbf{ [1]}$$

$$wlp(X := E, Q) = Q[E/X] \textbf{ [2]}$$

($Q[E/X]$ is the substitution of every occurrence of X in Q state by the value E)

Ivy example :

- Let's have a Q state containing $x = 4$ and $y = 8$
- $wlp(x := y, Q) = Q\{x = 8, y = 8\}$

$$wlp(s1; s2, Q) = wlp(s1, wlp(s2, Q)) \textbf{ [3]}$$

This rule explain why we are going backward when we use weakest precondition: the first WLP calculus rule that will be effectively applied will start by the last statement.

Ivy example :

- We want to prove that Y = 8.
- $wlp(x := 3; y = 5; y := x + y, Q) = Q = \{y = 8\}$
- $wlp(x := 3; y = 5, wlp(y := x + y, Q) = Q = \{y = 8\}$
- $wlp(x := 3; wlp(y = 5, wlp(y := x + y, Q)) = Q = \{y = 8\}$
- $wlp(x := 3, wlp(y = 5, Q)) = Q = \{x + y = 8\}$
- $wlp(x := 3; Q3) = Q = \{x + 5 = 8\}$
- $Q = \{8 = 8\}$

It is obvious that 8=8 we therefore proved that $y = 8$.

$$wlp(\text{if } C \text{ then } s1 \text{ else } s2, Q) =$$
$$(C \rightarrow wlp(s1, Q)) \wedge (\neg C \rightarrow wlp(s2, Q)) \textbf{ [4]}$$
$$wlp(\text{assume } S, Q) = (S \rightarrow Q) \textbf{ [5]}$$
$$wlp(\text{assert } S, Q) = (S \wedge Q) \textbf{ [6]}$$

B. Quantifier-free linear integer arithmetic

Algorithms that solve the problem of determining whether a given first-order logic formula are exponential in time for the worst cast since this problem is NP-complete.

These algorithms handle a theory called "Quantifier-Free Linear Integer Arithmetic" (QFLIA). [11] It considers a set of variable X only containing integers which allows us to form combination of constraints define with operators "`and`" ($\wedge$), "`or`" ($\vee$) and "`not`" ($\neg$).

It uses a kind of proof by contradiction where as reminder, to prove proposition $P \rightarrow Q$ then:

- We assume P to be false, i.e $\neg P$
- We have to show that $\neg P \rightarrow (Q \wedge \neg Q)$ which is a contradiction, so P must be true.

The algorithm proceed more or less the same way by searching if the negated proposition/condition has a solution or not meaning in the first case that the proposition is not valid.

C. Decidable fragment and Z3

First let's explain what is a decidable fragment. A fragment in mathematical logic is a subset of this logic theory which is obtained by adding restrictions on it. We have a reduction from a problem to another one.

Proceeding like that allow to take advantage from the property that the time complexity of saying whether a fragment is satisfiable/decidable or not cannot be higher than for the original problem.

Z3 SMT solver use this and given enough time and memory, it should always be able to determine if a formula in the fragment is satisfiable.

In practice, Z3 is very predictable for formula inside fragment than formula outside because it can easily have a reliable counter-models/negated proposition. It is also very efficient for small formulas as we can expected.

We also know that all quantifier-free formulas are in decidable fragment. This allow us to reason on formula with quantifier, for that let's take a look at a simple but good example of McMillan:

$$\forall X : f(X) > X$$

To prove this, we can instantiate the quantifier for some constant y like this:

$$f(y) > y$$

We know as a consequence of Herbrand's theorem that the method of instantiate quantifier is a complete method, so if we show that the verified condition is not satisfiable using instanciation, then this formula is unsatisfiable in general. But the inverse is not always true except for decidable fragment where it can be shown that there is always a finite set of instances such that is they are satisfiable then the formula is satisfiable.

VI. Implementation details

You can find all our changes in the following repositories [15] [16] and some examples that we made in [17] and that implement some concepts explain just before.

References

[1] Retrieved from http://microsoft.github.io/ivy/language.html
[2] Retrieved from https://www.eecs.berkeley.edu/ apanda/
[3] Retrieved from https://www.cs.tau.ac.il/ msagiv/
[4] http://www2.mta.ac.il/sharon.shoham/
[5] http://people.mpi-inf.mpg.de/mvoigt/files/Dissertation_with_hyperref.pdf
[6] https://github.com/Z3Prover/z3
[7] "Guarded commands, nondeterminacy and formal derivation of programs"
[8] Retrieved from http://microsoft.github.io/ivy/decidability.html
[9] https://utah.instructure.com/courses/362531/files/folder/lectures?preview=53847316
[10] https://www.lri.fr/ marche/MPRI-2-36-1/2013/poly1.pdf
[11] https://cs.nyu.edu/media/publications/king_tim.pdf [p105]
[12] https://github.com/microsoft/ivy
[13] https://link.springer.com/chapter/10.1007/978-3-030-53291-8_12 [p190]
[14] https://en.wikipedia.org/wiki/Propositional_calculus
[15] https://github.com/ElNiak/QUIC-Ivy/tree/quic_18_upgrade_chris/doc/examples/quic
[16] https://github.com/ElNiak/Toward-verification-of-QUIC-extensions
[17] https://github.com/ElNiak/QUIC-Ivy/tree/quic_18_upgrade_chris/doc/examples/quic/ivy_tests